Memory Reach: GRU vs MLP under PPO on MiniGrid Memory

Anh Khoa Pham
github.com/khoaphamanh/rl_project

Abstract

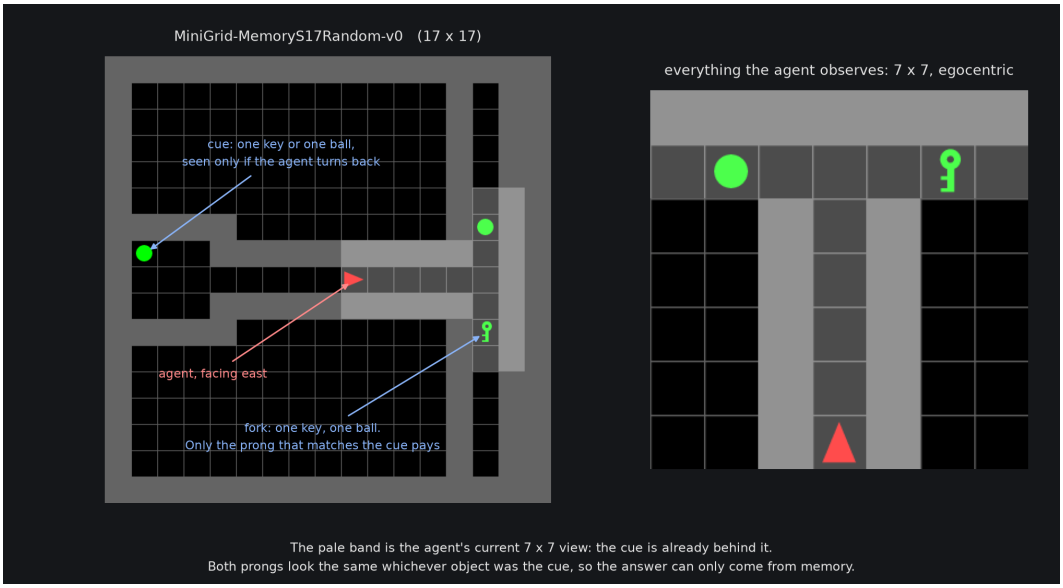
A recurrent policy can only use a memory that its gradient has taught it to write. We measure how far back that gradient has to reach on `MiniGrid-MemoryS17Random-v0`, a partially observable task in which a cue visible only at one end of a corridor decides which prong of a T-junction at the far end is correct. One PPO pipeline is run with a memoryless MLP and with a GRU whose truncated backpropagation-through-time (TBPTT) window L is varied over $\{1, 4, 8, \infty\}$, and each arm is tuned by its own 50-trial Optuna study over identical ranges at an identical fixed budget. The outcome is a sharp threshold between $L = 4$ and $L = 8$: $L \in \{8, \infty\}$ reach 1.000 ± 0.000 success, while $L \in \{1, 4\}$ and the MLP stay at chance. Carrying the hidden state forward is therefore not the deciding factor, since every GRU arm does that; the gradient itself has to span the interval between cue and decision. Truncating is also not cheaper in time: full BPTT was the best-scoring arm and the fastest GRU arm (1.3 h vs. 2.6 h at $L = 1$), and paid only in VRAM.

1 Introduction

In a partially observable task the optimal action depends on information that has left the observation [Kaelbling et al., 1998]. The usual remedy is a recurrent policy carrying a hidden state h_t forward [Hausknecht and Stone, 2015, Ni et al., 2022], and the usual practical concession is to truncate the backward pass to L steps [Williams and Peng, 1990, Kapturowski et al., 2019], because full backpropagation through time [Werbos, 1990] costs activation memory linear in sequence length. That concession is usually made silently, with L inherited from whatever codebase a project starts from. We ask what it buys and what it costs:

1. **How far back must the gradient reach** before a policy solves a task that a memoryless policy provably cannot? Carrying h_t forward is free regardless of L, so is a short window enough to *learn* what to store, or must the gradient span cue and decision?
2. **What does that reach cost** in wall-clock time and memory? A shorter window is intuitively cheaper; we check whether it is.

The setting is `MiniGrid-MemoryS17Random-v0` [Chevalier-Boisvert et al., 2023] (Figure 1, top). The agent sees a 7×7 egocentric patch, so at the junction the observation is identical whichever answer is correct: a memoryless policy is capped at chance, and the gap between 0.5 and 1.0 success is exactly the memory a run has acquired. We hold one PPO [Schulman et al., 2017] pipeline fixed and change only the encoder (MLP or GRU [Cho et al., 2014]) and, for the GRU, $L \in \{1, 4, 8, \infty\}$. Since hyperparameters and outcomes are tightly coupled in RL [Eimer et al., 2023, Andrychowicz et al., 2021], every arm gets an independent Optuna [Akiba et al., 2019] study over identical ranges and an identical non-searchable budget.



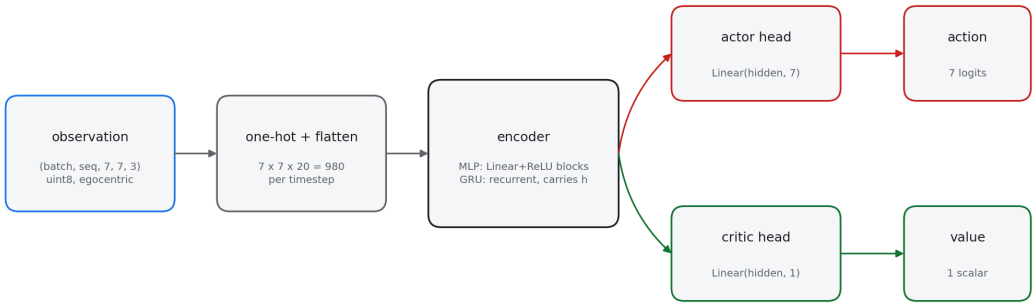


Figure 1: **Top:** `MiniGrid-MemoryS17Random-v0`. A cue (key or ball) sits in the west room; the corridor forks into a T with a key at one prong and a ball at the other, and success requires reaching the prong matching the cue. The agent spawns at a random corridor position facing away from the cue, so the information must be actively sought. **Bottom:** one shared encoder feeding a linear actor head over the 7 actions and a linear critic head. The encoder is the only part that differs between arms.

2 Related Work

Hausknecht and Stone [2015] showed that a recurrent value function recovers performance under flickering observations, and Ni et al. [2022] found that a carefully implemented recurrent model-free agent is a strong baseline across many POMDPs, with details such as how sequences are cut and how hidden states are seeded mattering more than the algorithm. Attention-based memories [Parisotto et al., 2020] and dedicated benchmarks [Morad et al., 2023] pursue the same question with larger machinery. Truncated BPTT [Williams and Peng, 1990] bounds the cost of training such policies, and R2D2 [Kapturowski et al., 2019] studies the artefacts it introduces, proposing the stored-state and burn-in recipe now in common use; our chunking follows the same stored-state principle, but where R2D2 fixes L and corrects for it, we vary L and ask where the task stops being solvable. Finally, Engstrom et al. [2020], Andrychowicz et al. [2021] document how much of the reported difference between deep RL methods comes from tuning rather than from the method, and Eimer et al. [2023] recommend the kind of per-method tuning protocol at a stated budget that we follow below.

3 Approach

Task and network. The environment is a POMDP: a $7 \times 7 \times 3$ egocentric view (object, colour and state index per cell), one-hot encoded per channel into 980 features, and 7 discrete actions. Reward is MiniGrid's $1 - 0.9 \cdot \text{step\_count}/\text{max\_steps}$ on success and 0 otherwise, with `max_steps` set to the rollout length $T = 361$. Both arms share the actor-critic skeleton of Figure 1 (bottom) and differ only in the encoder f_θ, which maps $(B, T, 7, 7, 3)$ to (B, T, H): the MLP applies $n \in \{1, \ldots, 4\}$ `Linear+ReLU` blocks to each timestep independently, so no state crosses the sequence and its gradient spans one step, while the GRU [Cho et al., 2014] is a single layer of width H carrying h_t forward, so an observation from a hundred steps ago can still be present at the decision.

PPO. One iteration collects $W = 32$ workers $\times$ $T = 361$ steps, computes advantages by GAE [Schulman et al., 2016],

$$\delta_t = r_t + \gamma V(s_{t+1})(1 - d_t) - V(s_t), \qquad \hat{A}_t = \delta_t + \gamma\lambda(1 - d_t)\hat{A}_{t+1}, \qquad \hat{R}_t = \hat{A}_t + V(s_t), \tag{1}$$

standardises $\hat{A}$ once over the whole rollout (returns $\hat{R}$ keep their own scale, being what the critic must predict), and takes up to $n_{\text{epochs}} = 3$ shuffled passes minimising

$$\mathcal{L} = -\,\mathbb{E}_t\Big[\min\Big(\rho_t\hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big] + c_v\mathbb{E}_t\Big[(V_\theta(s_t) - \hat{R}_t)^2\Big] - c_e\mathbb{E}_t[\mathcal{H}(\pi_\theta(\cdot \mid s_t))]\,, \tag{2}$$

with $\rho_t = \exp(\log \pi_\theta(a_t|s_t) - \log \pi_{\theta_{\text{old}}}(a_t|s_t))$ [Schulman et al., 2017]. Every expectation covers real timesteps only, since chunks are padded inside a minibatch and padding never enters the loss, and gradients are clipped to `max_grad_norm` before each Adam [Kingma and Ba, 2015] step.

The knob under study. Before the update the rollout is cut at every episode boundary *and* every L steps (Algorithm 1). A chunk beginning mid-episode is seeded with the hidden state the rollout already recorded there, so the forward pass still sees the whole episode and **only the backward pass is shortened**. This isolates the quantity of interest: all GRU arms carry identical information forward and differ only in how far credit assignment reaches. Setting $L = \infty$ (full BPTT) cuts at episode boundaries only. Note that a smaller L yields *more, shorter* sequences, which is why truncating does not automatically save time.

Algorithm 1 One PPO iteration with truncation length L. The highlighted line is the only difference between arms.

Require: policy π_θ, critic V_θ, workers W, steps T, truncation length L

1: $\mathcal{B} \leftarrow$ rollout of $W \times T$ transitions, recording $(o_t, a_t, \log \pi_t, V_t, r_t, d_t, h_t)$
2: compute $\hat{A}, \hat{R}$ by GAE (Eq. 1); standardise $\hat{A}$ over $\mathcal{B}$
3: $\mathcal{S} \leftarrow$ CHUNK($\mathcal{B}$): split at episode boundaries **and** every L steps, seeding each chunk with its recorded h
4: **for** epoch $= 1 \ldots n_{\text{epochs}}$, minibatch $\mathcal{M} \subset \mathcal{S}$ (shuffled, padded) **do**
5: forward f_θ over $\mathcal{M}$ from the stored h_0, evaluate Eq. 2 on non-padding steps
6: backpropagate ($\leq L$ steps through the recurrence), clip gradient norm, Adam step
7: **end for**

Tuning protocol. Each arm gets one Optuna study [Akiba et al., 2019] of 50 trials with a TPE sampler [Bergstra et al., 2011] seeded at 42. A trial trains the *same five seeds* $[0, 15, 12, 97, 98]$ for 1000 iterations ($\approx$ 11.6M environment steps per seed) and evaluates each on 50 fixed mazes from a held-out evaluation seed. Trials are ranked by $\text{mean}_s[\text{return}_s] - \text{std}_s[\text{return}_s]$, the spread taken across seeds and never across the episodes of one run, since the per-episode return is bimodal and within-run spread would merely restate the success rate. Nine parameters are searched over ranges identical for both encoders, plus the MLP's depth (Appendix A). Nothing that buys compute is searchable: epochs, iterations, workers, rollout length and L are fixed, and each L is its own study, so no sampler ever ranks one truncation length against another. The minibatch size is instead resolved per run by probing candidates from 4096 downwards and keeping the largest that fits in VRAM, so every run is measured at the same memory budget and the winning size is itself a result.

Table 1: Winning trial per study. *steps* is the mean evaluation episode length, *minibatch* the size the VRAM probe settled on, and *time* the wall-clock training time of that trial for all five seeds.

arm	encoder	return	success rate	steps	minibatch	time
`MLP`	MLP (memoryless)	0.556 ± 0.029	0.592 ± 0.027	24.3	1024	1.3 h
`GRU` L=1	GRU	0.573 ± 0.064	0.588 ± 0.068	9.8	4096	2.6 h
`GRU` L=4	GRU	0.492 ± 0.001	0.500 ± 0.000	7.0	4096	1.6 h
`GRU` L=8	GRU	$\mathbf{0.958 \pm 0.004}$	$\mathbf{1.000 \pm 0.000}$	16.8	4096	2.0 h
`GRU` full BPTT	GRU	$\mathbf{0.958 \pm 0.002}$	$\mathbf{1.000 \pm 0.000}$	16.8	512	**1.3 h**

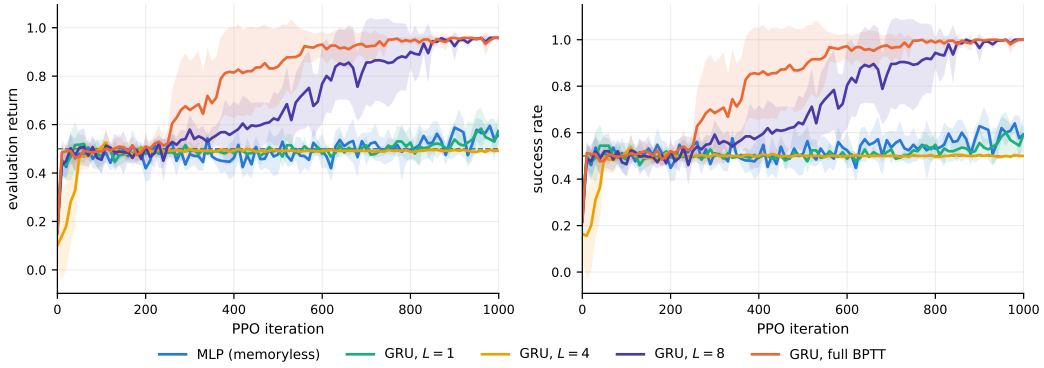


Figure 2: Training curves of the winning trial of each study: means over the five seeds, bands at ± 1 standard deviation across seeds, dashed line at chance (0.5). The two solving arms separate well before iteration 400; the three failing arms sit on the chance line for the whole budget.

4 Experiments

Setup. Five studies were run (MLP; GRU at $L \in \{1, 4, 8\}$ and full BPTT), that is 250 training runs and roughly 236 GPU-hours, on one NVIDIA RTX 3060 Laptop GPU (12 GB VRAM) with an AMD Ryzen 9 7950X, using PyTorch [Paszke et al., 2019], Gymnasium [Towers et al., 2024] and MiniGrid [Chevalier-Boisvert et al., 2023]. Every number below is the winning trial of that study, averaged over its five seeds, with $\pm$ denoting the standard deviation across seeds.

The cut-off is sharp, and eight steps suffice for a seventeen-step solution. Table 1 and Figure 2 answer the first question categorically rather than gradually: $L \in \{8, \infty\}$ solve the task at 1.000 success on all five seeds and return 0.958, while $L = 4$ (0.500), $L = 1$ (0.588) and the MLP (0.592) never beat a coin flip, and no setting produced an intermediate score such as 0.75. Carrying h forward is therefore not what matters, since every GRU arm does that identically, including the ones that fail; what separates them is whether the gradient reaches back to the observation that filled h, and reaching further than 8 adds nothing, as full BPTT matches $L = 8$ within noise. Why eight is enough is not explained by the 16.8-step episode, which is the wrong comparison: truncation never limits how long the policy can *remember*, only whether the encoder can *learn to write* the cue into h, which requires the cue observation and the rewarded decision to fall inside the same chunk. The quantity that must fit inside L is thus d_{informed}, the fewest actions from spawn to a correct decision for an agent that goes and looks at the cue first. Measuring d_{informed} by breadth-first search over the environment's own state and visibility model, on 2000 mazes redrawn at every reset (`hallway_end` $\in \{4, \ldots, 14\}$), gives $d_{\text{informed}} \leq 4$ in 0.0% of mazes, ≤ 8 in 29.6%, and ≤ 16 in 93.9% (mean 11.0, median 11, minimum 5, maximum 21). The floor of 5 actions – one turn to see the cue plus turn-back, forward, turn, forward to the correct prong – is hit in only 99 of the 2000 mazes, those where the agent spawns exactly at the junction mouth, so no maze at all is solvable, informed, within four actions: $L = 4$ has zero mazes where the gradient can ever pair the cue with the reward, while $L = 8$ reaches a real, if thin, 29.6% slice of the short tail. Replaying the trained checkpoints over the 50 evaluation mazes confirms it: cue and decision share a chunk in 16% of the $L = 8$ agent's episodes and in 0 of 50 of the $L = 4$ agent's. Partial coverage suffices because a GRU applies the same weights at every timestep,

so the write rule acquired on a 6-step episode transfers to a 20-step one at no cost, and because GAE, computed over the full rollout before the split (Eq. 1), carries reward credit across chunk boundaries irrespective of L. The failing arms are correspondingly not partially remembering: at 7.0 steps the $L = 4$ arm dashes straight to the fork and always picks the same side, which scores exactly $25/50$ on this evaluation set, so its across-seed spread of 0.000 is arithmetic rather than convergence. That memory-free optimum is easy to reach – a blind 3-action dash reaches the fork within four actions in 25.9% of mazes – since the detour costs reward immediately and pays nothing until the memory works, and the trained $L = 4$ checkpoint stops looking almost entirely, seeing the cue in just 11 of 50 episodes.

The reach costs memory, not time. Full BPTT is the fastest GRU arm at 1.3 h, matching the memoryless MLP, against 1.6 h ($L = 4$), 2.0 h ($L = 8$) and 2.6 h ($L = 1$). The intuition that truncation saves time is wrong at fixed rollout size: sequences per epoch scale as $\lceil T/L \rceil$, so halving L roughly doubles the number of padded, kernel-launch-bound minibatch sequences, and the per-sequence saving in the backward pass does not compensate (the extremes are the clean comparison, as the middle two arms also differ in width). Full BPTT is earlier as well, reaching 0.95 success at a median of 370 iterations against 640 for $L = 8$. What it pays is VRAM: the probe settles at a minibatch of 512 against 4096 for $L = 8$, an $8\times$ difference on the same 12 GB GPU. The recommendation on this task is therefore full BPTT, with $L = 8$ as the fallback on a smaller GPU; below $L = 8$ one does not get a cheaper solution but the MLP's guessing policy at GRU prices.

5 Discussion

On MemoryS17Random the gradient must reach back between 4 and 8 steps, with no partial credit on either side of that boundary and no benefit beyond it, and the reach costs activation memory ($8\times$ smaller minibatches) but not wall-clock time, since hard truncation was the slowest configuration we ran. The control that makes this interpretable is the stored-state chunking: every GRU arm carries the identical hidden state forward, so the collapse at $L \leq 4$ is attributable to credit assignment and not to information availability. The per-arm tuning protocol was essential rather than cosmetic, since with 50 trials each and a variance-penalised objective the failure of $L \in \{1, 4\}$ cannot be dismissed as an unlucky learning rate. Our cost intuition did not survive: we expected a score/compute trade-off along L and found the best-scoring setting to be the cheapest in time as well, so truncation is best understood here as a memory concession rather than a speed optimisation.

Limitations and future work. This is one task, one recurrent architecture and one algorithm. The threshold is a property of MemoryS17Random, and we did not resolve $L \in \{5, 6, 7\}$, so it is bracketed rather than located. Five seeds separate 0.5 from 1.0 safely but do not support fine claims such as the 0.556 versus 0.573 ordering of two chance-level arms, and evaluation uses 50 fixed mazes from a single seed. Three extensions follow directly: sweep $L \in \{5, 6, 7\}$ and control the corridor length directly instead of leaving it to the environment's randomisation, which would locate the threshold rather than bracket it and test the d_{informed} account of Section 4 causally; add R2D2-style burn-in [Kapturowski et al., 2019], which would tell us whether a short window can be made to behave like a long one, in which case the threshold is about optimisation rather than the gradient path itself; and repeat the protocol with an LSTM [Hochreiter and Schmidhuber, 1997] and a small transformer [Parisotto et al., 2020], where the analogous knob is the attention window.

References

Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD 2019)*, pages 2623–2631, 2019.

Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphaël Marinier, Léonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, Sylvain Gelly, and Olivier Bachem. What matters for on-policy deep actor-critic methods? a large-scale study. In *9th International Conference on Learning Representations (ICLR 2021)*, 2021.

James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems 24 (NIPS 2011)*, pages 2546–2554, 2011.

Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo Perez-Vicente, Lucas Willems, Salem Lahlou, Suman Pal, Pablo Samuel Castro, and Jordan Terry. Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023.

Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP 2014)*, pages 1724–1734, 2014.

Theresa Eimer, Marius Lindauer, and Roberta Raileanu. Hyperparameters in reinforcement learning and how to tune them. In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, pages 9104–9149, 2023.

Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. Implementation matters in deep policy gradients: A case study on PPO and TRPO. In *8th International Conference on Learning Representations (ICLR 2020)*, 2020.

Matthew J. Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable MDPs. In *2015 AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*, 2015.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8): 1735–1780, 1997.

Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998.

Steven Kapturowski, Georg Ostrovski, John Quan, Rémi Munos, and Will Dabney. Recurrent experience replay in distributed reinforcement learning. In *7th International Conference on Learning Representations (ICLR 2019)*, 2019.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR 2015)*, 2015.

Steven Morad, Ryan Kortvelesy, Matteo Bettini, Stephan Liwicki, and Amanda Prorok. POPGym: Benchmarking partially observable reinforcement learning. In *11th International Conference on Learning Representations (ICLR 2023)*, 2023.

Tianwei Ni, Benjamin Eysenbach, and Ruslan Salakhutdinov. Recurrent model-free RL can be a strong baseline for many POMDPs. In *Proceedings of the 39th International Conference on Machine Learning (ICML 2022)*, pages 16691–16723, 2022.

Emilio Parisotto, H. Francis Song, Jack W. Rae, Razvan Pascanu, Caglar Gulcehre, Siddhant M. Jayakumar, Max Jaderberg, Raphaël Lopez Kaufman, Aidan Clark, Seb Noury, Matthew M. Botvinick, Nicolas Heess, and Raia Hadsell. Stabilizing transformers for reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML 2020)*, pages 7487–7498, 2020.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, pages 8024–8035, 2019.

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *4th International Conference on Learning Representations (ICLR 2016)*, 2016.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017. URL `https://arxiv.org/abs/1707.06347`.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. *CoRR*, abs/2407.17032, 2024. URL `https://arxiv.org/abs/2407.17032`.

Paul J. Werbos. Backpropagation through time: What it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 1990.

Ronald J. Williams and Jing Peng. An efficient gradient-based algorithm for on-line training of recurrent network trajectories. *Neural Computation*, 2(4):490–501, 1990.

A Search space and winning hyperparameters

Table 2 gives the search space, identical for both encoders, and the configuration of the winning trial of each study, that is the settings behind every number in Table 1. The tuned values are similar across the solving and the failing GRU arms (learning rates within a factor of 1.4, comparable discounting), so the outcome is not explained by one study finding a special corner of the space. The one visible pattern is the MLP's much larger entropy coefficient and value weight, consistent with a run that never finds a deterministic solution to commit to.

Table 2: Search space (identical ranges per encoder) and the winning configuration per study. `score` is $\mathrm{mean}_s - \mathrm{std}_s$ of the evaluation return over the five seeds, the quantity Optuna maximised. Fixed, never-searched settings: $n_{\text{epochs}} = 3$, 1000 iterations, 32×361 rollout, and L itself.

parameter	range	MLP	L=1	L=4	L=8	full BPTT
winning trial		3	47	40	25	36
`lr`	10^{-5}–10^{-2}, log	$6.65{\cdot}10^{-4}$	$1.53{\cdot}10^{-3}$	$1.48{\cdot}10^{-3}$	$1.35{\cdot}10^{-3}$	$1.82{\cdot}10^{-3}$
`gamma`	0.99–0.9999	0.9917	0.9968	0.9962	0.9940	0.9968
`gae_lambda`	0.9–0.99	0.90	0.95	0.90	0.98	0.96
`clip_eps`	0.1–0.3	0.26	0.19	0.10	0.13	0.25
`entropy_coef`	10^{-4}–10^{-1}, log	$7.03{\cdot}10^{-2}$	$9.27{\cdot}10^{-4}$	$3.30{\cdot}10^{-4}$	$5.11{\cdot}10^{-3}$	$9.56{\cdot}10^{-3}$
`value_coef`	0.01–1.0, log	0.854	0.120	0.028	0.128	0.089
`wd`	10^{-8}–10^{-2}, log	$6.72{\cdot}10^{-7}$	$1.60{\cdot}10^{-7}$	$2.74{\cdot}10^{-7}$	$1.21{\cdot}10^{-8}$	$2.87{\cdot}10^{-8}$
`max_grad_norm`	0.1–2.0, log	0.134	1.152	1.226	0.304	0.500
`hidden_size`	32–512, step 8	360	216	360	480	280
`n_layers_mlp`	1–4 (MLP only)	2				
`score`		0.527	0.509	0.491	0.955	**0.956**

Checklist

This checklist is not mandatory, but can help you set up your project in a reproducible manner. Options for filling it out are: [Yes] [No] [NA]

1. General points:
   (a) Do the main claims made in the abstract and introduction accurately reflect your contributions and scope? [Yes] The claims are limited to one task, one recurrent architecture and the four truncation lengths actually run.
   (b) Did you cite all relevant related work? [Yes] See Section 2 (recurrent policies in POMDPs, truncated BPTT and stored states, tuning protocols in RL).
   (c) Did you describe the limitations of your work? [Yes] Section 5: single task and architecture, threshold bracketed rather than located, five seeds, one evaluation seed.
   (d) Did you include a discussion of future work? [Yes] Section 5, three concrete extensions.
2. If you are including theoretical results...
   (a) Did you state the full set of assumptions of all theoretical results? [NA] The report is purely empirical.
   (b) Did you include complete proofs of all theoretical results? [NA]
3. If you ran experiments (e.g. for benchmarks)...
   (a) Did you include the code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL)? [Yes] Code, the exact command per study, and the finished Optuna studies with the winning checkpoints are at `https://github.com/khoaphamanh/rl_project`.
   (b) Did you specify all the training details (e.g., data splits, hyperparameters, how they were chosen)? [Yes] Protocol in Section 3; search space and selected values in Appendix A.
   (c) Did you run at least 10 repetitions of your method? [No] Each configuration was trained on five seeds $[0, 15, 12, 97, 98]$, the same five in every trial of every study, for 250 training runs and $\approx$ 236 GPU-hours. The measured effect (0.500 vs. 1.000 success, across-seed std ≤ 0.004 on the solving arms) is far larger than the seed noise, so more repetitions would not change the conclusion; a finer comparison between the two chance-level arms would need them.
   (d) Did you report error bars (e.g., with respect to the random seed after running experiments multiple times)? [Yes] All tables report mean $\pm$ std across seeds, Figure 2 shades ± 1 std, and the tuning objective is itself mean minus one std across seeds.
   (e) Did you include the total amount of compute and the type of resources used (e.g., type of GPUs, internal cluster, or cloud provider)? [Yes] Section 4: one RTX 3060 Laptop GPU (12 GB), Ryzen 9 7950X, $\approx$ 236 GPU-hours in total.
4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets...
   (a) If your work uses existing assets, did you cite the creators? [Yes] MiniGrid [Chevalier-Boisvert et al., 2023], Gymnasium [Towers et al., 2024], Optuna [Akiba et al., 2019] and PyTorch [Paszke et al., 2019] are cited where used.
   (b) Did you make sure the license of the assets permits usage? [Yes] All four are permissively licensed (MIT/BSD-style) and used unmodified.
   (c) Did you reference the assets directly within your code and repository? [Yes] They are pinned in `requirements.txt` and named in the repository README.